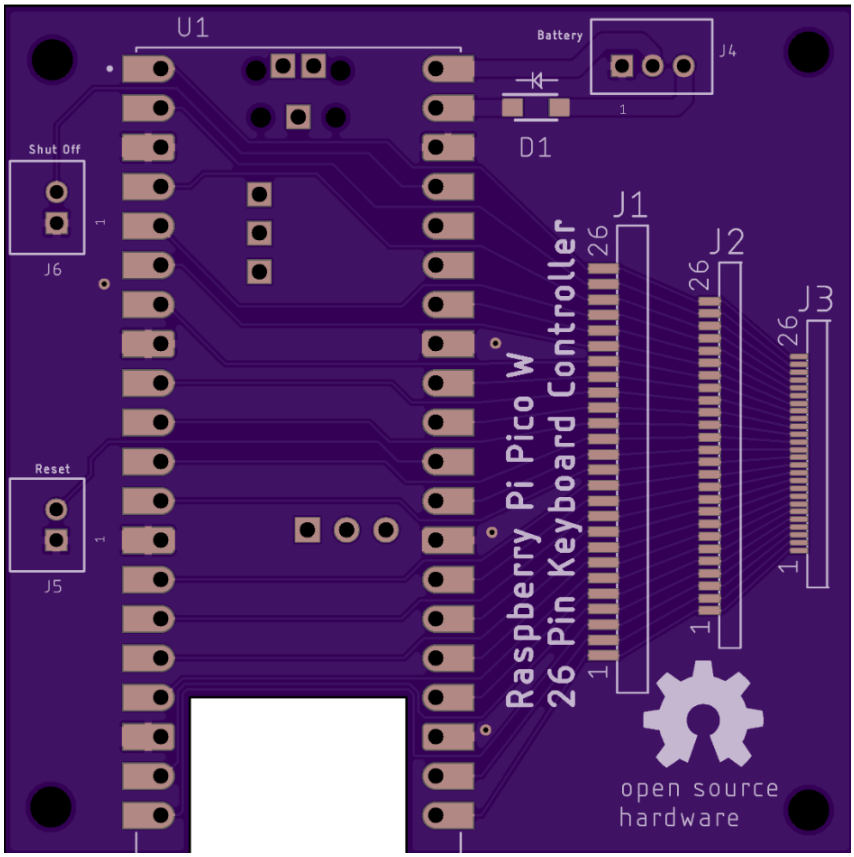
Raspberry Pi Pico W 26 Pin Keyboard Controller

This document will describe how to make a USB or BLE controller for a typical laptop keyboard with 26 or less FPC pins using a Raspberry Pi Pico (W) that uses the RP2040 controller IC. The USB coding can be done with Arduino C++ code, KMK, or QMK. Note that the Pi Pico 2 uses the RP2350 controller, which is not yet supported by QMK. The BLE coding is done with Arduino C++ code and will work on the Pi Pico W and the Pi Pico 2 W.

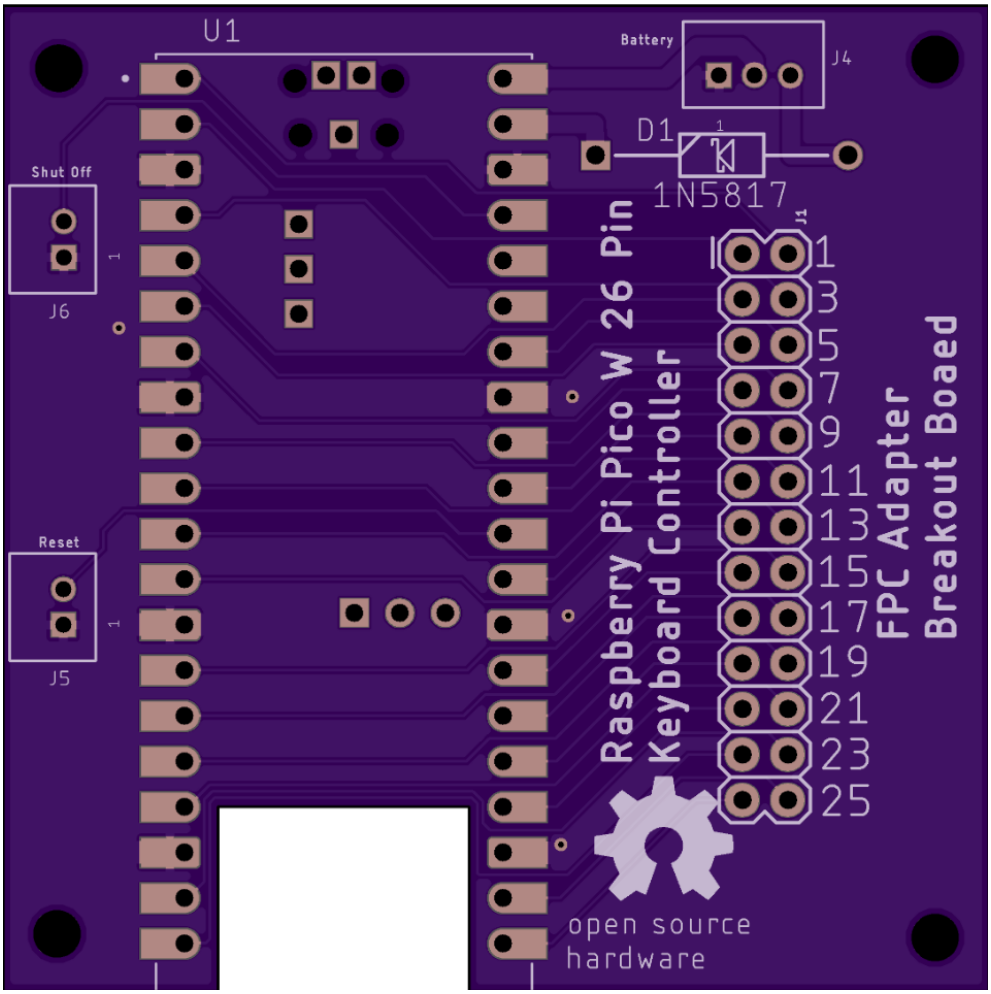
If your keyboard has more than 26 pins, you can use the RP2350 Stamp XL documented at my repo but it will only work as a KMK USB controller, not QMK. I have developed a Pi Pico (W) circuit board that controls an MCP23018 port expander for keyboards with up to 40 FPC pins. The PDF documentation for the port expander board is at my repo.

The 26 pin Pi Pico circuit board shown below has pads to solder either a 1.0mm, 0.8mm, or 0.5mm pitch FPC connector. All associated files are at my Github repository.



The cutout in the board gives better reception for the Bluetooth antenna on the Pico. The Pico can be mounted with header pins or soldered directly to the board for a lower profile. If the keyboard will only be used for USB, it will always have power. The Schottky diode and the 3 pin JST battery connector at J4 are not needed. The 2 pin JST connector at J6 is only needed if you are running under battery power and want to power down the Pico. The 2 pin JST connector at J5 is only needed if you want a reset push button switch. The "Pico_26Pin_Keyboard.brd" Eagle file and "Pico_26Pin_Keyboard.zip" Gerber file at my repo can be fabricated by OSH Park or other fab houses like JLCPCB.

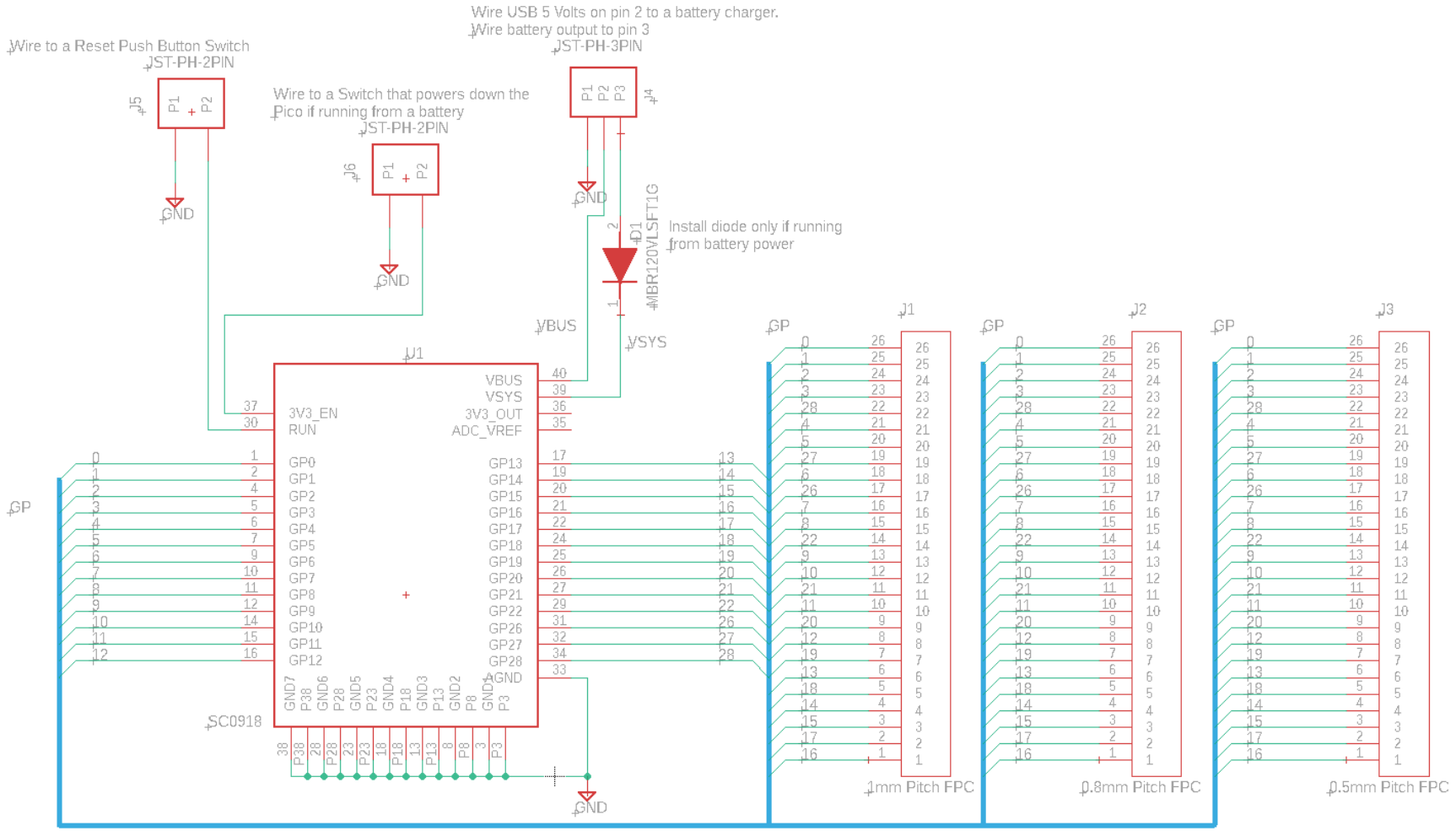
I designed a second connector board for the Pico that uses a standard FPC adapter board. The through hole pins on the adapter are much easier to solder compared to the fine pitch surface mount pins on the FPC connector. The traces from the Pico to the FPC connector are the same as the first board so no change is needed for the software. The "Pico_26Pin_FPC_Adapter.brd" Eagle file and "Pico_26Pin_FPC_Adapter.zip" Gerber file can be downloaded from my repo.



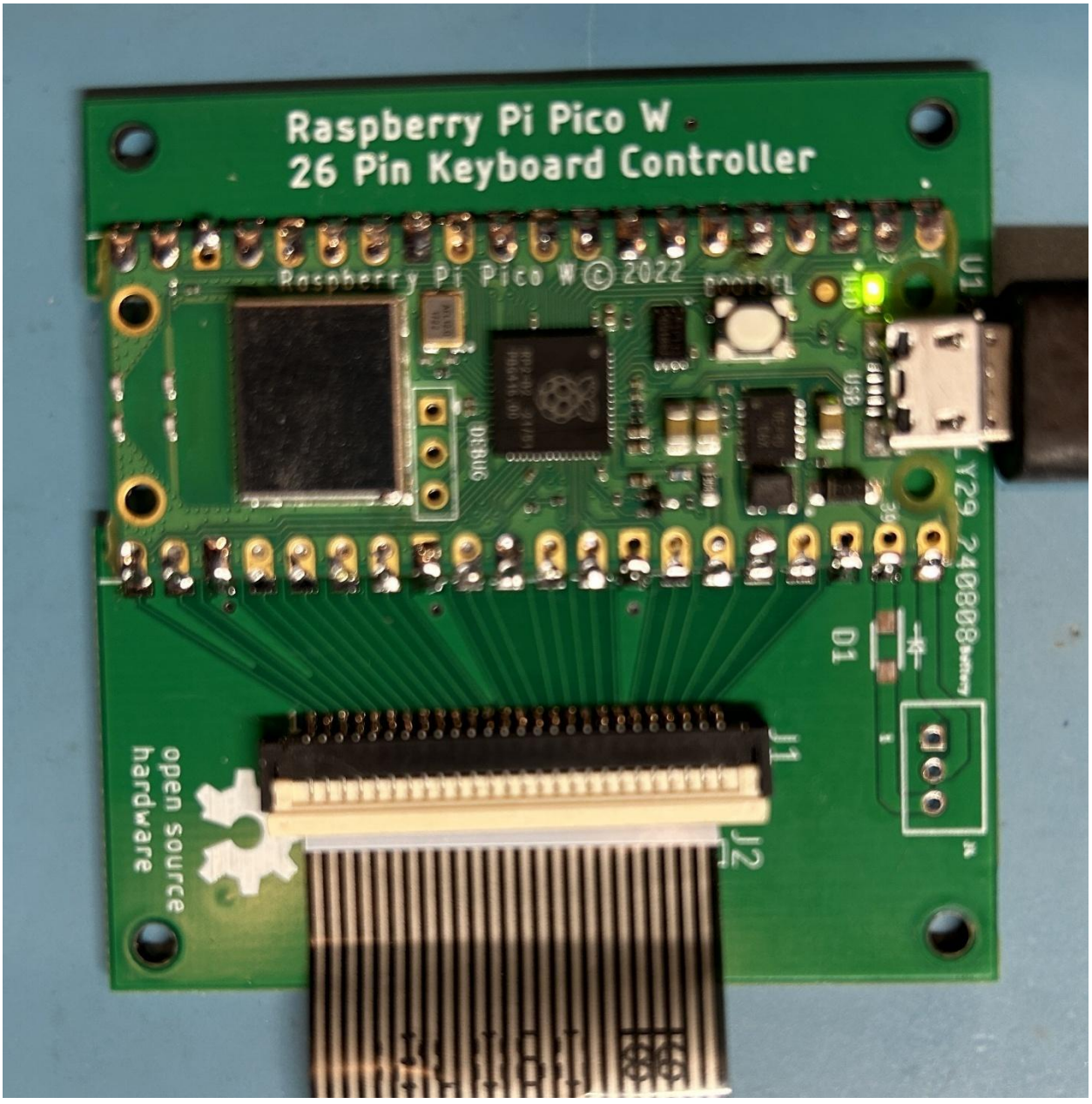
Both circuit boards measure 55mm x 55mm (2.17" x 2.17").

Bluetooth Low Energy (BLE) operation implies the controller will be running from a separate lithium battery with a charging circuit. USB 5 volts on the Pico VBUS pin is routed to pin 2 on J4 which is a 3 pin JST connector. This should be cabled (along with ground) to an off-board battery charger circuit. The nominal 3.7 volt battery output should be cabled to the JST connector J4 pin 3. This feeds an MBR120VLSFT1G or 1N5817 Schottky diode that "or-ties" the battery voltage to the Pico's VSYS pin. There is another Schottky diode in the Pico that brings USB power to VSYS when the USB cable is attached.

The Eagle schematic "Pico_26Pin_Keyboard.sch" at my repo is shown below. The schematic for the "Pico_26Pin_FPC_Adapter" is also at my repo.


Wire USB 5 Volts on pin 2 to a battery charger.
Wire battery output to pin 3
Wire to a Reset Push Button Switch
JST-PH-2PIN
JST-PH-3PIN
Wire to a Switch that powers down the Pico if running from a battery
JST-PH-2PIN
J5
J6
J4
J1
J2
J3
P1
P2
P3
GND
D1
MBR120VLSFT1G
Install diode only if running from battery power
VBUS
VSYS
U1
3V3_EN
RUN
3V3_OUT
ADC_VREF
AGND
GP0
GP1
GP2
GP3
GP4
GP5
GP6
GP7
GP8
GP9
GP10
GP11
GP12
GP13
GP14
GP15
GP16
GP17
GP18
GP19
GP20
GP21
GP22
GP26
GP27
GP28
GND7
P38
GND6
P28
GND5
P23
GND4
P18
GND3
P13
GND2
P8
GND1
P3
SC0918
GP
1mm Pitch FPC
0.8mm Pitch FPC
0.5mm Pitch FPC

The assembled circuit board with a Raspberry Pi Pico W and a 26 pin 1mm pitch FPC connector is shown below. I soldered this Pico directly to the connector board for a low profile but it could also be soldered with header pins. This is an early version of the board that didn't have the 2 pin JST connectors for Reset and Enable switches. The FPC cable from a Dell Inspiron 1545 keyboard is attached. The procedure to make this example keyboard operational over USB using KMK and QMK will be described first followed by the procedure to make a USB and BLE controller using Arduino C++ code.



Once your Pico and FPC connector are soldered to the circuit board, you need to determine how the key matrix for your keyboard is wired. This will be done using my matrix decoder CircuitPython program running on the Pico.

Go to circuitpython.org/ and select the “Get Started” box to install CircuitPython. The following is a synopsis of the steps that they provide.

To enter into boot loader mode, hold the Pico push button down while plugging in the Pico’s USB cable. The new folder will show as a drive named RPI-RPI2.

If the Pico is acting weird or you want to start with a clean slate, copy the flash_nuke.uf2 file (downloaded here) over to the Pico’s RPI-PPI2 folder. The drive will blink out and then return all clean.

Python - Start with the circuitpython.org/downloads page, then select your Pico model. Download and copy the latest Adafruit Circuit Python uf2 over to the Pico’s RPI-PPI2 folder. The drive will blink out and return as CIRCUITPY with an empty lib folder. The code.py file is a one-line “Hello world” program. You will be replacing this file later.

Keyboard library – Go to circuitpython.org/libraries and download the library bundle that matches the version of Python you are using. Only copy the Adafruit_HID folder into the lib folder on the Pico.

As you write and debug your Python code you will need an editor. I prefer Thonny and it can be downloaded at thonny.org/. Thonny has a built in Python interpreter and a fresh Thonny install defaults to running the Python code locally on your PC, not on the Pico. To make Thonny run the code on the Pico, go to the bottom right corner of Thonny and click Local Python 3 and then "configure interpreter". In the first drop down, select CircuitPython (generic). Leave everything else checked and then select OK. Now you can select stop and then run in Thonny and it runs the code on the Pico.

Download my Matrix_Decoder.py code, then in Thonny, select File, Open, This Computer and navigate to the Matrix_Decoder.py code you just downloaded. This will bring it up in the editor and you can read through the comments to get an idea of how it works. On Thonny, select the “Stop” icon to halt the program that is running on the Pico. Next select File, Save As, CircuitPython Device. Overwrite the existing code.py file with your Thonny code. Select the “Run” icon so the Pico will run the code.py program. As you debug your code, you will need to hit the Stop icon in order to save your changes to the Pico. Be sure to also save your updated code to your computer. If the program sends two numbers when you select “Run”, before any keys are pressed, these are probably grounds or LEDs in the keyboard that must be skipped by removing them from the I_O array starting on line 33 in the code. Once you have no numbers reported when you hit "Run", you're ready to make a key matrix.

Bring up another editor like Notepad++ and load a blank keyboard_pin_list.txt file that lists all the possible keyboard keys. This particular list uses KMK names. If you are making a QMK controller, use keyboard_pin_list_qmk_blank.txt instead. Place the cursor to the far right of the first key in the list. This is where the Python program will send the Pico GPIO numbers when you push a key. Then the program will send a down arrow to position the cursor for the next key. Install the keyboard FPC cable in the connector and push each key on the list that exists on your keyboard. Use your PC mouse or arrow keys to skip over any keys you don’t have.

Dell 1545 Example Keyboard Connected to the Pico



Once you have a complete list, there are 3 ways you can determine the row and column pins and populate the key matrix:

Method 1 - Use your favorite AI assistant (like ChatGPT, Claude, or Gemini) to analyze your filled-out keyboard pin list text file. To force the AI to accurately find the hidden matrix geometry without creating a giant, un-optimized grid, copy and paste the following prompt. Note: For QMK, change *KMK firmware framework* to QMK. Also replace the KMK main.py section with *QMK keymaps.c with QMK keycodes, and list the row and column GPIO's for the keyboard.json file.*

AI Prompt: "I am building a USB laptop keyboard controller using a Raspberry Pi Pico and the KMK firmware framework. I have attached my raw keyboard GPIO pin connection list below. Because laptop matrices lack schematics and omit diodes, I do not know the exact number of rows and columns. Analyze the overlapping intersections in my pin list to isolate the optimized, condensed hardware matrix geometry. The column axis will represent a small group of shared lines (typically 8 or 9 pins), while the intersecting row axis will hold the remaining unique pins. Organize these into an efficient row_pins and col_pins configuration (do not output a giant, un-optimized square array). Finally, output a complete, production-ready KMK main.py file with the pins mapped and the keyboard.keymap nested array fully populated with the matching kmk keycodes. Here is my raw pin connection file: ***[INSERT YOUR FILE HERE or PROVIDE A LINK]****".*

Method 2 is to run Marcel's program that I've modified for the Pico. This semi-automated program will determine the rows, columns, and key matrix. There will still be some hand editing but most of the hard work is done. My PDF describes how to use Marcel's program.

Method 3 is to use the same manual procedure used on a Teensy microcontroller from steps 12 and 13 of my Instructable. I used this method for the Dell example keyboard to determine which GPIO's are rows and which are columns. A quick run-through using the Modifier keys; Control, Alt, Shift, GUI, and Fn is given below. Usually there will be 8 GPIO's that will be columns and programmed as Pico inputs with pull ups. The remaining GPIO's will be rows and driven low one at a time by the Pico. The remaining rows will be floated so they don't interfere. Refer to the Dell Inspiron 1545 keyboard pin connections at my repo to follow along with the methodology given below:

Control-Left and Control-Right have GPIO 8 in common which will be a row. GPIO's 2 and 27 will be columns. Shift-Left and Shift-Right have GPIO 22 in common (row) and GPIO's 2 and 5 are columns (we already knew 2 was a column). Alt-Left and Alt-Right have GPIO 9 in common (row) and GPIO's 1 and 6 are columns. So far, we know 1, 2, 5, 6, and 27 are columns. GPIO 4 on the GUI key would make sense as a column based on looking down the list. Likewise, GPIO 3 for the Fn key makes sense as a column when looking down the list. That gives 7 columns and to find the 8th, look down the list to find a key that doesn't use 1, 2, 3, 4, 5, 6, or 27. The E key shows 28 is the last column. All other GPIO's will be rows. Build a key matrix like the one shown below with the column GPIO's listed across the top and all the remaining GPIO's listed as rows down the side. Now place each key name at the appropriate row/column intersection (see example matrix tables below).

Dell 1545 Keyboard Matrix

	GP1	GP2	GP3	GP4	GP5	GP6	GP27	GP28
GP7			PGDOWN	LGUI				PGUP
GP8		RCTRL					LCTRL	
GP9	RALT			PSCREEN		LALT		EJCT
GP10		Z	A	N1	TAB	ESC	GRAVE	Q
GP11		C	D	N3	F3	F4	F2	E
GP12	SPACE	ENTER	BSLASH	F10	BSPACE	F5	F9	
GP13		COMMA	K	N8	RBRC	F6	EQUAL	I
GP14		DOT	L	N9	F7		F8	O
GP15	LEFT			END		UP	HOME	
GP16	RIGHT			F12			INSERT	
GP17	DOWN			F11			DELETE	
GP18	SLASH		SCOLON	N0	LBRC	QUOTE	MINUS	P
GP19	N	M	J	N7	Y	H	N6	U
GP20	B	V	F	N4	T	G	N5	R
GP21		X	S	N2	CAPS		F1	W
GP22		RSHIFT			LSHIFT			
GP26			FN					

The table below is the same as above except the media keys replace certain function keys. These are sent when Fn is held down.

	GP1	GP2	GP3	GP4	GP5	GP6	GP27	GP28
GP7			PGDOWN	LGUI				PGUP
GP8		RCTRL					LCTRL	
GP9	RALT			PSCREEN		LALT		EJCT
GP10		Z	A	N1	TAB	ESC	GRAVE	Q
GP11		C	D	N3	F3	BRID	F2	E
GP12	SPACE	ENTER	BSLASH	MPRV	BSPACE	BRIU	VOLU	
GP13		COMMA	K	N8	RBRC	F6	EQUAL	I
GP14		DOT	L	N9	MUTE		VOLD	O
GP15	LEFT			END		UP	HOME	
GP16	RIGHT			MNXT			INSERT	
GP17	DOWN			MPLY			DELETE	
GP18	SLASH		SCOLON	N0	LBRC	QUOTE	MINUS	P
GP19	N	M	J	N7	Y	H	N6	U
GP20	B	V	F	N4	T	G	N5	R
GP21		X	S	N2	CAPS		F1	W
GP22		RSHIFT			LSHIFT			
GP26			FN					

At this point, you will create either KMK or QMK files or C++ code for a USB or BLE keyboard controller.

KMK USB Controller:

KMK is open-source firmware normally used for mechanical keyboards but it also works for laptop keyboards, as long as you know how the switches are wired (see above tables). KMK uses CircuitPython which is why I also wrote my matrix decoder code in CircuitPython. All of the hard work of scanning key switches and communicating over USB is embedded in KMK. All you need to do is edit my KMK Dell example code with your key matrix information. Getting started with KMK is described at their GitHub repo which I will detail below.

For step 1, you already have CircuitPython installed.

For step 2, click on "get an up-to-date copy of KMK".

For step 3, unzip the kmk_firmware-main folder. The top folder is kmk_firmware-main. It has many folders and files. Copy the KMK folder to the CIRCUITPY drive on the Pico.

For step 4, use my code_Dell1545.py as a starting point. Load it into Thonny and edit it as follows:

Change keyboard.col_pins to list the GPIO column pins left to right across the top of your matrix.

Change keyboard.row_pins to list the GPIO row pins top to bottom along the side of your matrix.

The COL2ROW setting for the keyboard.diode_orientation will force KMK to use open-drain scanning to protect diodeless matrices like those found in laptop keyboards.

Change the keyboard.keymap matrix for the base layer and the Fn media layer per the tables you created. Use the KMK basic key names and the media key names. Each keycode name (except FN) must be preceded with KC. Put KC.NO in any cell of the matrix that has no key at that location.

After saving the code to your computer, save it to the Pico's CIRCUITPY drive with the name code.py (this will overwrite your previous code.py that was the matrix decoder).

In Thonny, click the "Stop" icon, then click the "Run" icon. You should now be able to bring up a blank editor like Notepad++ and type on the keyboard to test if all the keys work. If you have any typo's, they will be reported in the Thonny console window. Use the keyboardchecker.com website to verify all the function keys work.

For normal USB keyboard operation (without Thonny), you can just plug in the USB cable. This will bring up a CIRCUITPY drive folder which you can close.

QMK USB Controller: (Note QMK does not support the Raspberry Pi Pico 2 or any other RP2350 module)

QMK is widely considered the de facto open-source standard firmware for mechanical keyboards. There are lots of custom printed circuit boards for building a QMK based mechanical keyboard but very little for laptop keyboard control. The QMK C software allows up to 32 layers with advanced macros, RGB lighting, and "tap-dance" keys. I'm just a beginner when it comes to QMK but my simplified build procedure can be expanded to add many more features.

Read through the QMK tutorial and other web sites to get a feel for the build process and the command line interface (CLI). The CLI uses Python scripts so you'll need a standalone version of Python installed on your computer (not just the Python bundled inside Thonny). A good code editor like Notepad++ is also needed. Disregard talk about the browser based "QMK Configurator" as it's not useful for a laptop keyboard.

Download and run QMK-MSYS. Use its CLI to issue the command, "qmk setup". The installer will download a lot of files from GitHub including a folder full of mechanical keyboard files (that you won't use). The tutorials tell you to build test code for one of the mechanical keyboards in order to see if you have all the files and tools properly set up. You can skip this or use my MacBook A1286 files to see if there's a problem.

Use the pwd, ls, and cd Linux commands in the CLI to navigate to the following directory:

/c/Users/<user_name>

In the CLI, run the "qmk new-keyboard" command. This will create files and folders in the proper directory structure for later editing. It will ask for your new keyboard name, which can only have lower case letters, underscore, and numbers. Also, it will ask for your GitHub user name and real name, (leave these blank if you want). Select "none of the above" from the list of mechanical keyboards. Select "No" when asked if the controller is on a development board. Select the RP2040 for a Raspberry Pi Pico. Laptop keyboards typically don't have diodes but you must make the diode_direction line be COL2ROW. A new folder with your keyboard name will be created in the keyboards directory. On my Windows 11 PC, it was at:

/c/Users/<user_name>/qmk_firmware/keyboards/<keyboard_name>

Navigate to the new <keyboard_name> folder and it will have 2 files, keyboard.json and readme.md as well as one folder named keymaps.

Use your editor to modify the keyboard.json file. The extrakey setting needs to be "true" if you are adding an Fn media key matrix. The rows and cols matrix_pins are determined by one of the 3 methods described earlier. The RP2040 I/O pins are listed without quotes or GP ahead of the number (per the latest QMK version). A portion of the dell1545 keyboard.json file for a RP2040 is shown below, (complete file at my repo). The diode_direction is defined as COL2ROW to force open-drain scanning of the rows which protect key matrices like laptops that have no diodes.

"manufacturer": "Frank Adams",

"keyboard_name": "dell1545",

"maintainer": "thedalles77",

```
"bootloader": "rp2040",
"diode_direction": "COL2ROW",
"features": {
"bootmagic": false,
"extrakey": true,
"mousekey": false,
"nkro": false
},
"matrix_pins": {
"rows": [16, 17, 15, 14, 18, 13, 19, 12,
20, 11, 21, 10, 9, 22, 8, 7, 26],
"cols": [6, 27, 5, 4, 28, 3, 2, 1]
},
"processor": "RP2040",
"url": "",
"usb": {
"device_version": "1.0.0",
"pid": "0x0000",
"vid": "0xFEED"
},
"layouts": {
"LAYOUT": {
"layout": [
{"matrix": [0, 0], "x": 0, "y": 0},
{"matrix": [0, 1], "x": 1, "y": 0},
{"matrix": [0, 2], "x": 2, "y": 0},
{"matrix": [0, 3], "x": 3, "y": 0},
{"matrix": [0, 4], "x": 4, "y": 0},
{"matrix": [0, 5], "x": 5, "y": 0},
```

{"matrix": [0, 6], "x": 6, "y": 0},

{"matrix": [0, 7], "x": 7, "y": 0},

This pattern repeats until all keys are defined. The number of rows times the number of cols determines how many “matrix” lines are required. For the Dell 1545 example, there are 17 rows X 8 cols = 136 total. The last group of 8 pins in the keyboard.json file is given below. There are 17 groups of 8 to give 136 total.

{"matrix": [16, 0], "x": 0, "y": 16},

{"matrix": [16, 1], "x": 1, "y": 16},

{"matrix": [16, 2], "x": 2, "y": 16},

{"matrix": [16, 3], "x": 3, "y": 16},

{"matrix": [16, 4], "x": 4, "y": 16},

{"matrix": [16, 5], "x": 5, "y": 16},

{"matrix": [16, 6], "x": 6, "y": 16},

{"matrix": [16, 7], "x": 7, "y": 16}

Note; there is no comma after the last matrix line.

The next step is to edit the keymap.c file found two levels down at:

/c/Users/<your_user_name>/qmk_firmware/keyboards/<keyboard_name>/keymaps/default

Use the matrix table you created to populate the first array with the base layer keys. The Dell 1545 keymaps.c file shown below is at my repo.

```
#include QMK_KEYBOARD_H
const uint16_t PROGMEM keymaps[][MATRIX_ROWS][MATRIX_COLS] = {
    [0] = LAYOUT(
        KC_NO,   KC_INS,  KC_NO,   KC_F12,  KC_NO,   KC_NO,   KC_NO,   KC_RGHT,
        KC_NO,   KC_DEL,  KC_NO,   KC_F11,  KC_NO,   KC_NO,   KC_NO,   KC_DOWN,
        KC_UP,   KC_HOME, KC_APP,  KC_END,  KC_NO,   KC_NO,   KC_NO,   KC_LEFT,
        KC_NO,   KC_F8,   KC_F7,   KC_9,    KC_O,    KC_L,    KC_DOT,  KC_NO,
        KC_QUOT, KC_MINS, KC_LBRC, KC_0,    KC_P,    KC_SCLN, KC_NO,   KC_SLSH,
        KC_F6,   KC_EQL,  KC_RBRC, KC_8,    KC_I,    KC_K,    KC_COMM, KC_NO,
        KC_H,    KC_6,    KC_Y,    KC_7,    KC_U,    KC_J,    KC_M,    KC_N,
        KC_F5,   KC_F9,   KC_BSPC, KC_F10,  KC_NO,   KC_BSLS, KC_ENT,  KC_SPC,
        KC_G,    KC_5,    KC_T,    KC_4,    KC_R,    KC_F,    KC_V,    KC_B,
        KC_F4,   KC_F2,   KC_F3,   KC_3,    KC_E,    KC_D,    KC_C,    KC_NO,
        KC_NO,   KC_F1,   KC_CAPS, KC_2,    KC_W,    KC_S,    KC_X,    KC_NO,
        KC_ESC,  KC_GRV,  KC_TAB,  KC_1,    KC_Q,    KC_A,    KC_Z,    KC_NO,
        KC_LALT, KC_NO,   KC_NO,   KC_PSCR, KC_NO,   KC_NO,   KC_NO,   KC_RALT,
        KC_NO,   KC_NO,   KC_LSFT, KC_NO,   KC_NO,   KC_NO,   KC_RSFT, KC_NO,
        KC_NO,   KC_LCTL, KC_NO,   KC_NO,   KC_NO,   KC_NO,   KC_RCTL, KC_NO,
        KC_NO,   KC_NO,   KC_NO,   KC_LGUI, KC_PGUP, KC_PGDN, KC_NO,   KC_NO,
        KC_NO,   KC_NO,   KC_NO,   KC_NO,   KC_NO,   MO(1),   KC_NO,   KC_NO
    ),
```

The second array with the media keys that are activated when the Fn key is held down is given next.

```
[1] = LAYOUT(
    KC_NO,   KC_INS,  KC_NO,   KC_MNXT, KC_NO,   KC_NO,   KC_NO,   KC_RGHT,
    KC_NO,   KC_DEL,  KC_NO,   KC_F11,  KC_NO,   KC_NO,   KC_NO,   KC_DOWN,
    KC_UP,   KC_HOME, KC_APP,  KC_END,  KC_NO,   KC_NO,   KC_NO,   KC_LEFT,
    KC_NO,   KC_VOLD, KC_MUTE, KC_9,    KC_O,    KC_L,    KC_DOT,  KC_NO,
    KC_QUOT, KC_MINS, KC_LBRC, KC_0,    KC_P,    KC_SCLN, KC_NO,   KC_SLSH,
    KC_F6,   KC_EQL,  KC_RBRC, KC_8,    KC_I,    KC_K,    KC_COMM, KC_NO,
    KC_H,    KC_6,    KC_Y,    KC_7,    KC_U,    KC_J,    KC_M,    KC_N,
    KC_BRIU, KC_VOLU, KC_BSPC, KC_MPRV, KC_NO,   KC_BSLS, KC_ENT,  KC_SPC,
    KC_G,    KC_5,    KC_T,    KC_4,    KC_R,    KC_F,    KC_V,    KC_B,
    KC_BRID, KC_F2,   KC_F3,   KC_3,    KC_E,    KC_D,    KC_C,    KC_NO,
    KC_NO,   KC_F1,   KC_CAPS, KC_2,    KC_W,    KC_S,    KC_X,    KC_NO,
    KC_ESC,  KC_GRV,  KC_TAB,  KC_1,    KC_Q,    KC_A,    KC_Z,    KC_NO,
    KC_LALT, KC_NO,   KC_NO,   KC_PSCR, KC_NO,   KC_NO,   KC_NO,   KC_RALT,
    KC_NO,   KC_NO,   KC_LSFT, KC_NO,   KC_NO,   KC_NO,   KC_RSFT, KC_NO,
    KC_NO,   KC_LCTL, KC_NO,   KC_NO,   KC_NO,   KC_NO,   KC_RCTL, KC_NO,
    KC_NO,   KC_NO,   KC_NO,   KC_LGUI, KC_PGUP, KC_PGDN, KC_NO,   KC_NO,
    KC_NO,   KC_NO,   KC_NO,   KC_NO,   KC_NO,   KC_TRNS, KC_NO,   KC_NO
)
```

QMK basic keycodes start with KC_ followed by the key name. Use KC_NO at any array location that has no key. Put MO(1) at the Fn position in the base layer and KC_TRNS at the Fn position in the second layer. If you want additional layers, there are lots of tutorials that give examples.

After editing keyboard.json and keymaps.c, use the CLI to navigate to /c/Users/<user_name>/qmk_firmware/keyboards/<keyboard_name>

Compile the code in the CLI with the command:

qmk compile -kb <your_keyboard_name> -km default

The compile takes 30 minutes on my slow computer (once I clear the typo's and syntax errors). The QMK source code is updated every 3 months so depending on what version you downloaded from their repo, you may need to change the syntax of the commands shown here. When this happens to me, I give the compile errors and my files to Google and let the AI tell me what's wrong.

Flashing your controller:

To load your new QMK code into a Raspberry Pi Pico, start with the USB disconnected, push and hold the button on the Pico while plugging in the USB cable. Release the button and a folder will show up as RPI-RPI2 with a drive letter. Drag the <keyboard_name>_default.uf2 file that was just compiled into the Pico folder. The .uf2 file is located at C:Users/<user_name>/qmk_firmware

The RPI-RPI2 folder will blink out and the keyboard should be functional. Unlike KMK, there is no folder that opens whenever you plug in the USB cable.

Bluetooth Low Energy (BLE)

KMK and QMK do not support BLE on the Pi Pico W but it is supported by the Earle Philhower Arduino C++ libraries. You will need to load the Pi Pico W or Pi Pico 2 W libraries into the Arduino IDE. Modify the C++ code with your key matrix and row/column GPIO numbers. In the Arduino IDE, set the board to Raspberry Pi Pico W or Pico 2 W and the IP/Bluetooth Stack to IPV4 + Bluetooth. Each time you re-flash the Pico, you need to "Remove Device" in your computer or phone's Bluetooth settings and then pair it again.

The Bluetooth Low Energy (BLE) Arduino code for the Dell 1545 keyboard is at my repo. The section you will need to change is shown below.

```
const int NUM_ROWS = 17; // number of row signals
const int NUM_COLS = 8; // number of column signals

// column and row GPIO numbers attached to the column and row pins of the keyboard
const int colPins[NUM_COLS] = {1, 2, 3, 4, 5, 6, 27, 28};
const int rowPins[NUM_ROWS] = {7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 26};

// Base Keyboard Map. A zero indicates no key at that position.
const uint8_t keyMap[NUM_ROWS][NUM_COLS] = {
  { 0,              0,              KEY_PAGE_DOWN,  KEY_LEFT_GUI,     0,              0,              0,              KEY_PAGE_UP },
  { 0,              KEY_RIGHT_CTRL, 0,              0,                0,              0,              KEY_LEFT_CTRL,  0 },
  { KEY_RIGHT_ALT,  0,              0,              KEY_PRINT_SCREEN, 0,              KEY_LEFT_ALT,   0,              0 },
  { 0,              'z',            'a',            '1',              KEY_TAB,        KEY_ESC,        '`',            'q' },
  { 0,              'c',            'd',            '3',              KEY_F3,         KEY_F4,         KEY_F2,         'e' },
  { ' ',            KEY_RETURN,     '\\',           KEY_F10,          KEY_BACKSPACE,  KEY_F5,         KEY_F9,         0 },
  { 0,              ',',            'k',            '8',              ']',            KEY_F6,         '=',            'i' },
  { 0,              '.',            'l',            '9',              KEY_F7,         0,              KEY_F8,         'o' },
  { KEY_LEFT_ARROW, 0,              0,              KEY_END,          0,              KEY_UP_ARROW,   KEY_HOME,       0 },
  { KEY_RIGHT_ARROW,0,              0,              KEY_F12,          0,              0,              KEY_INSERT,     0 },
  { KEY_DOWN_ARROW, 0,              0,              KEY_F11,          0,              0,              KEY_DELETE,     0 },
  { '/',            0,              ';',            '0',              '[',            '\'',           '-',            'p' },
  { 'n',            'm',            'j',            '7',              'y',            'h',            '6',            'u' },
  { 'b',            'v',            'f',            '4',              't',            'g',            '5',            'r' },
  { 0,              'x',            's',            '2',              KEY_CAPS_LOCK,  0,              KEY_F1,         'w' },
  { 0,              KEY_RIGHT_SHIFT,0,              0,                KEY_LEFT_SHIFT, 0,              0,              0 },
  { 0,              0,              0,              0,                0,              0,              0,              0 }
};
```

The GPIO numbers as well as the key locations in the matrix were determined with the matrix decoder CircuitPython code described earlier. Put a 0 in any location that has no key. Use the names above as guides for what you need to put in your new matrix. A few special cases:

'\\' this is the backspace key

'\'' this is the single quote key

Change the Bluetooth name associated with your keyboard at line 57:

KeyboardBLE.begin("Dell1545 Keyboard", "Dell Mod");

The Fn - media keys have not yet been coded.

Arduino C++ USB Code

You will need to load the Adafruit TinyUSB Library into the Arduino IDE. Under Tools, USB Stack, select Adafruit TinyUSB. The example USB code for a Dell 1545 laptop keyboard is at my repo. Modify this code for your keyboard by determining the key connections using the Circuit Python code from step 16 and then building the key matrix (either manually or with the help of AI).

Use the "Keyboard_Pin_List_TinyUSB_Blank.txt" file for the correct TinyUSB key names. Modify the following sections of C++ code with your key matrix and row/column GPIO numbers.

const int NUM_ROWS = 17; // number of row signals

const int NUM_COLS = 8; // number of column signals

// column and row GPIO numbers attached to the column and row pins of the keyboard

const int colPins[NUM_COLS] = {1, 2, 3, 4, 5, 6, 27, 28};

const int rowPins[NUM_ROWS] = {7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 26};

```
const uint8_t keyMap[NUM_ROWS][NUM_COLS] = {
  { 0,                  0,                    HID_KEY_PAGE_DOWN,  HID_KEY_GUI_LEFT,     0,                    0,                  0,                    HID_KEY_PAGE_UP },
  { 0,                  HID_KEY_CONTROL_RIGHT, 0,                 0,                     0,                    0,                  HID_KEY_CONTROL_LEFT, 0 },
  { HID_KEY_ALT_RIGHT,  0,                    0,                  HID_KEY_PRINT_SCREEN, 0,                    HID_KEY_ALT_LEFT,   0,                    0 },
  { 0,                  HID_KEY_Z,            HID_KEY_A,          HID_KEY_1,            HID_KEY_TAB,          HID_KEY_ESCAPE,     HID_KEY_GRAVE,        HID_KEY_Q },
  { 0,                  HID_KEY_C,            HID_KEY_D,          HID_KEY_3,            HID_KEY_F3,           HID_KEY_F4,         HID_KEY_F2,           HID_KEY_E },
  { HID_KEY_SPACE,      HID_KEY_ENTER,        HID_KEY_BACKSLASH,  HID_KEY_F10,          HID_KEY_BACKSPACE,    HID_KEY_F5,         HID_KEY_F9,           0 },
  { 0,                  HID_KEY_COMMA,        HID_KEY_K,          HID_KEY_8,            HID_KEY_BRACKET_RIGHT, HID_KEY_F6,        HID_KEY_EQUAL,        HID_KEY_I },
  { 0,                  HID_KEY_PERIOD,       HID_KEY_L,          HID_KEY_9,            HID_KEY_F7,           0,                  HID_KEY_F8,           HID_KEY_O },
  { HID_KEY_ARROW_LEFT, 0,                    0,                  HID_KEY_END,          0,                    HID_KEY_ARROW_UP,   HID_KEY_HOME,         0 },
  { HID_KEY_ARROW_RIGHT,0,                    0,                  HID_KEY_F12,          0,                    0,                  HID_KEY_INSERT,       0 },
  { HID_KEY_ARROW_DOWN, 0,                    0,                  HID_KEY_F11,          0,                    0,                  HID_KEY_DELETE,       0 },
  { HID_KEY_SLASH,      0,                    HID_KEY_SEMICOLON,  HID_KEY_0,            HID_KEY_BRACKET_LEFT, HID_KEY_APOSTROPHE, HID_KEY_MINUS,        HID_KEY_P },
  { HID_KEY_N,          HID_KEY_M,            HID_KEY_J,          HID_KEY_7,            HID_KEY_Y,            HID_KEY_H,          HID_KEY_6,            HID_KEY_U },
  { HID_KEY_B,          HID_KEY_V,            HID_KEY_F,          HID_KEY_4,            HID_KEY_T,            HID_KEY_G,          HID_KEY_5,            HID_KEY_R },
  { 0,                  HID_KEY_X,            HID_KEY_S,          HID_KEY_2,            HID_KEY_CAPS_LOCK,    0,                  HID_KEY_F1,           HID_KEY_W },
  { 0,                  HID_KEY_SHIFT_RIGHT,  0,                  0,                    HID_KEY_SHIFT_LEFT,   0,                  0,                    0 },
  { 0,                  0,                    0,                  0,                    0,                    0,                  0,                    0 }
};
```

The above key matrix uses the key names required by the Adafruit TinyUSB library.

The Fn - media keys are not yet coded.